WEEK 6 — SAMPLING FROM DISTRIBUTIONS 2: SLICE AND NESTED SAMPLING

Evidence = marginalized likelihood

MCMC REFRESHER

We construct a Markov chain whose stationary distribution is our target $p(x)$.
Metropolis–Hastings: propose $x' \sim q(x' \mid x_t)$, then accept with probability

$$\alpha(x' \mid x_t) = \min\left(1, \frac{p(x')}{p(x_t)} \frac{q(x_t \mid x')}{q(x' \mid x_t)}\right).$$

On rejection, keep $x_{t+1} = x_t$. The normalisation of p cancels.

CLICKER: REVISIT LAST WEEK'S QUESTION

Our target is $p(x) \propto e^{-x^2/2}$ and our proposal is

$$x' = x_t + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, 1).$$

If $x_t = 0$ and $x' = 1$, what is the acceptance probability?

- A. 1
- B. $e^{-1/2} \approx 0.607$
- C. $1/2$

Why can we omit the proposal probabilities here?

Answer: B. The random-walk proposal is symmetric:

$$q(x' \mid x_t) \propto e^{-(x' - x_t)^2/2} = q(x_t \mid x').$$

Thus

$$r = \frac{p(1)}{p(0)} = e^{-1/2}, \qquad \alpha = e^{-1/2}.$$

Going back from 1 to 0 is accepted with probability 1.

A DIFFERENT PROPOSAL GIVES A DIFFERENT ANSWER

The confusion last week was between a random walk and an **independence proposal**.
If we instead draw $x' \sim \mathcal{N}(0, 1)$ independently of x_t, the proposal equals the normalised target:

$$q(x' \mid x_t) = p(x'), \qquad q(x_t \mid x') = p(x_t).$$

Then

$$r = \frac{p(x')}{p(x_t)} \frac{p(x_t)}{p(x')} = 1.$$

Every proposal is accepted. This works because we can already sample the target directly!

WHY DO WE NEED THE HASTINGS CORRECTION?

Suppose our target is uniform on $[0, 1]$, but we propose independently from $q(x) = \mathcal{N}(x; 0, 1)$.
For $x_t, x' \in (0, 1)$,

$$r = \frac{p(x')}{p(x_t)} \frac{q(x_t)}{q(x')} = \frac{q(x_t)}{q(x')}.$$

Discuss: if we omitted this proposal ratio and accepted every proposal inside $[0, 1]$, which distribution would we sample?

A truncated Gaussian, rather than a uniform distribution. Proposals favour points near 0 over points near 1.
The ratio $q(x_t)/q(x')$ corrects this imbalance: moves towards more frequently proposed points can be rejected.
Proposals outside $[0, 1]$ are rejected, and we keep the current state.

TWO STATES: BUILD A CHAIN WE CAN SOLVE

Target: $\pi = (\rho,\, 1-\rho)^{\mathsf{T}}$ for states 0 and 1.
Always propose the other state, so the proposal ratio is 1. Assume $1/2 < \rho < 1$.
Let $a_{i \leftarrow j}$ denote acceptance of a proposal from j to i:

$$a_{0\leftarrow 1} = \min\left(1, \frac{\rho}{1-\rho}\right) = 1, \qquad a_{1\leftarrow 0} = \min\left(1, \frac{1-\rho}{\rho}\right) = \frac{1-\rho}{\rho}.$$

It is easier to move towards the more probable state.

THE TRANSITION MATRIX INCLUDES REJECTIONS

Use columns for the current state and rows for the next state:
$T_{ij} = \Pr(X_{t+1} = i \mid X_t = j)$.
Writing $a = (1-\rho)/\rho$,

$$T = \begin{pmatrix} 1-a & 1 \\ a & 0 \end{pmatrix}, \qquad p^{t+1} = T p^t.$$

Each column sums to 1. Rejected $0 \to 1$ proposals leave us in state 0.
Check detailed balance:

$$T_{10}\pi_0 = a\rho = 1-\rho = T_{01}\pi_1.$$

STATIONARY DOES NOT YET MEAN CONVERGED

The target is stationary: $T\pi = \pi$.
The other eigenvector is $v = (1, -1)^\mathsf{T}$, with eigenvalue

$$\lambda_2 = -a = -\frac{1-\rho}{\rho}.$$

Any initial probability vector can be written as $p^0 = \pi + cv$, where $c = p_0^0 - \rho$. Therefore

$$p^t = T^t p^0 = \pi + c\lambda_2^t v.$$

For $1/2 < \rho < 1$, $|\lambda_2| < 1$: the initial-condition term decays and $p^t \to \pi$.

EXAMPLE: $\rho = 3/4$

Starting entirely in state 0,

$$T = \begin{pmatrix} 2/3 & 1 \\ 1/3 & 0 \end{pmatrix}, \qquad \lambda_2 = -1/3.$$

Step t	$\Pr(X_t = 0)$	$\Pr(X_t = 1)$
0	1	0
1	$2/3$	$1/3$
2	$7/9$	$2/9$
3	$20/27$	$7/27$
∞	$3/4$	$1/4$

The negative eigenvalue gives alternating deviations from the target.

WHAT IF $\rho = 1/2$?

Then $T = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$: the chain alternates forever.

- The target $(1/2, 1/2)^\mathsf{T}$ is still stationary.
- From a fixed starting state, the distribution at step t **does not converge**.
- The fraction of visits to each state still tends to $1/2$.

This chain is periodic. Allowing a probability of staying put, for example $T_{\text{lazy}} = (I + T)/2$, removes the alternation.

Takeaway: detailed balance gives stationarity; convergence from arbitrary starts also needs the chain to explore the support and avoid periodicity.

THE 100-BOX CHALLENGE: CHOOSE YOUR JUMP

100 numbered boxes each contain 0–1000 CHF. You may inspect a proposed box; the money stays in place.
You are at box **20**, containing **900 CHF**. A nearby peak is around box 22.
In groups: would you propose a small jump or a large jump? What determines whether it is accepted?
Our sampling goal is $p(i) \propto m_i$, where m_i is the money in box i.

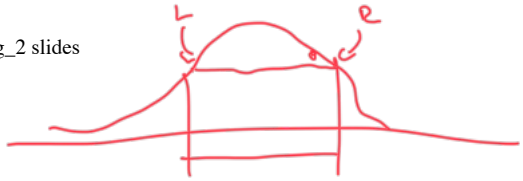
For a symmetric proposal, $\alpha = \min(1, m_{i'}/m_i)$.

Proposed box	Money	Acceptance from box 20
22	1000 CHF	1
50	10 CHF	$1/90$
70	850 CHF	$17/18$

Small jumps explore locally; large jumps can reach another peak but often land in poor boxes. Sampling asks for the right **visit frequencies**, rather than just the richest box.

SLICE SAMPLING

Slice sampling is another MCMC method.
It samples the region under an unnormalised target using an auxiliary height and a horizontal update.
Stepping-out and shrinking adapt the search interval to local width, reducing the reliance on a well-tuned MH proposal. The initial width still affects computational cost.



Slice sampling proceeds as follows:

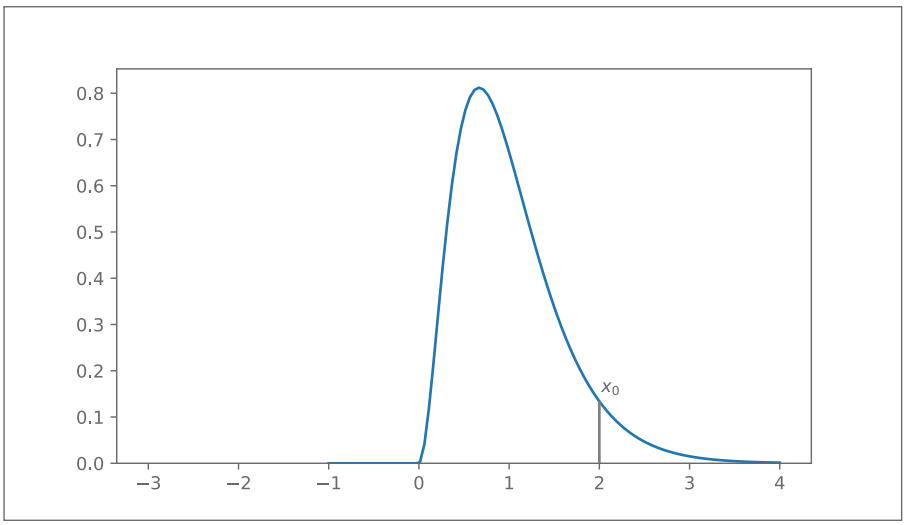
1. Sample u uniformly between 0 and $p(x_t)$: $u \sim \mathcal{U}(0, p(x_t))$
2. Find an interval $L < x_t < R$ such that $p(L) < u$ and $p(R) < u$
3. Draw x' uniformly from the interval $[L, R]$: $x' \sim \mathcal{U}(L, R)$
    - If $p(x') \leq u$, shrink the interval and return to 3.
    - If $p(x') > u$, the point (x', u) lies under the curve $p(x)$, so accept x': $x_{t+1} = x'$

- Finding the interval for step 2 uses a stepping out procedure:
  Given a step size w (this is the tuning parameter of slice sampling)
  1. Draw $r \sim \mathcal{U}(0,1)$
  2. Set $L = x_t - rw$, $R = x_t + (1-r)w$
  3. While $p(L) > u$: $L = L - w$
  4. While $p(R) > u$: $R = R + w$

- Shrinking the interval in step 3:
  1. While $p(x') \leq u$
     - If $x' > x_t$, $R = x'$
     - Else $L = x'$

-> useful for low dimensions, computationally expensive for high N, since normally need to draw N-1 slices

- does not need tuning
  -> step size does affect approximation of posterior -> always correct
  -> step size does affect sampling time





○ Once ◉ Loop ○ Reflect

```python
def sample_slice_sampling(n, x0, target_distr, step):
    p0 = target_distr.pdf(x0)

    for _ in range(n):
        u = np.random.uniform(0, p0)

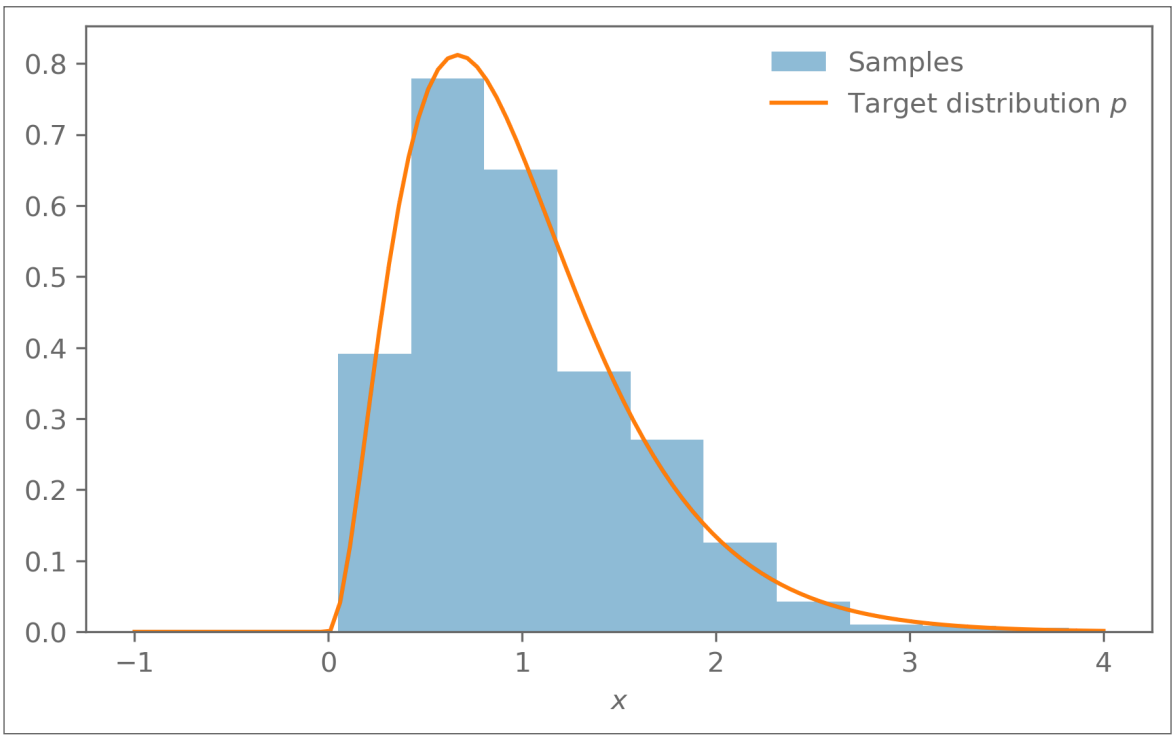
        # Define the initial interval
        w = np.random.uniform(0, 1)
        x_l, x_r = x0 - w*step, x0 + (1-w)*step

        # Define a step-out function
        def step_out(x, left=True):
            p = target_distr.pdf(x)
            while p > u:
                if left:
                    x -= step
                else:
                    x += step
                p = target_distr.pdf(x)
            return x

        # Step out until p(x_l) < u and p(x_r) < u
        x_l, x_r = step_out(x_l, left=True), step_out(x_r, left=False)

        # Shrink the interval until we have a sample from under the
        # curve of the target distribution
        while True:
            x1 = np.random.uniform(low=x_l, high=x_r)
            p1 = target_distr.pdf(x1)
            if p1 > u:
                # Accept the point x1
                break
            else:
                # Shrink the inverval
                if x1 >= x0:
                    x_r = x1
                else:
                    x_l = x1

        x0 = x1
        p0 = p1
        yield x0
```


Samples
Target distribution p
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
−1
0
1
2
3
4
x

THE SAME BOXES: TAKE A HORIZONTAL SLICE

Back at box 20 with 900 CHF, draw a height $u \sim \text{Uniform}(0, 900)$.
Suppose $u = 800$ **CHF**. Which boxes are eligible?

1. Are boxes 22 (1000), 50 (10), and 70 (850 CHF) in the slice?
2. Does a local stepping-out search necessarily find every eligible box?
3. What changes if we lower the slice height?

Eligible boxes satisfy $m_i > 800$: 22 and 70 qualify; 50 does not.
An ideal draw uniform over **all** eligible boxes can jump between peaks. Local stepping-out and shrinking may explore only one component.
Lower slices can connect peaks through valleys. Slice sampling adapts to local width, but does not guarantee efficient travel between separated modes.

Slice sampling is for example implemented in **zeus**, which uses a set of walkers like emcee to do the sampling.
Slice sampling is also being used under the hood in many nested sampling implementations.

CLICKER

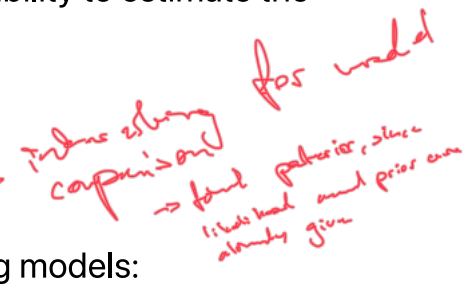
Which of the following is a key advantage of slice sampling over the Metropolis-Hastings algorithm?

- It does not require any tuning parameters.
- It automatically adjusts the step size to achieve optimal mixing.
- It requires less tuning.
- It guarantees faster convergence to the target distribution.

NESTED SAMPLING

A good review paper is **https://arxiv.org/abs/2205.15570**, which gives a good introduction as well as summarises the state of the art in nested sampling.
Nested sampling takes a very different approach to sampling than the MCMC methods covered so far. The main advantage is its ability to estimate the evidence $p(d)$ in Bayes' theorem:

$$p(\theta|d) = \frac{p(d|\theta)p(\theta)}{p(d)}$$



The evidence becomes very useful when comparing models:
Recall that we can use the Bayes' ratio to compare the probability of two models M_1 and M_2:

$$R = \frac{p(d|M_1)}{p(d|M_2)}$$

With nested sampling we can compute the evidences $p(d|M_1)$ and $p(d|M_2)$.

To make the notation clearer (and consistent with some of the literature on nested sampling), write this as

$$p(\theta) = \frac{L(\theta)\pi(\theta)}{Z} \,,$$

where

- $L(\theta) = p(d|\theta)$ is the likelihood,
- $\pi(\theta) = p(\theta)$ is the prior,
- $Z = p(d)$ is the evidence or marginal likelihood,
- $p(\theta)$ is the posterior.

How to compute the evidence?
Since the posterior is a normalised probability distribution:

$$\int p(\theta|d)p(d)\mathrm{d}\theta = \int p(d|\theta)p(\theta)\mathrm{d}\theta = p(d)\,.$$

Or in the other notation:

$$Z = \int L(\theta)\pi(\theta)\mathrm{d}\theta$$

For high-dimensional problems this is usually intractable by naive integration.
To see this, imagine discretising the integral into 50 intervals:
$Z = \sum_i^{50} L(\theta_i)\pi(\theta_i)\Delta\theta$. In 10 dimensions (which is not much as far as real-world applications are concerned), this would require $50^{10} \approx 10^{17}$ evaluations of the likelihood.

The idea behind nested sampling is to rewrite the integral so that instead of integrating over θ, the integral is over levels of the likelihood. This is somewhat like doing Lebesgue integration instead of Riemann integration.
Define $\mathrm{d}X = \pi(\theta)\mathrm{d}\theta$. Then

$$Z = \int L(\theta)\pi(\theta)\mathrm{d}\theta = \int L(X)\mathrm{d}X\,.$$

$L(X)$ is the inverse of $X(L)$, which is the volume of the likelihood (weighted by the prior) above some likelihood level L^*:

$$X(L^*) = \int_{L(\theta)>L^*} \pi(\theta)\mathrm{d}\theta$$

With this, the integral for Z can now be evaluated numerically, since $X \in (0,1)$, for example with the trapezoidal rule:

$$Z \approx \sum_i w_i L_i, \quad w_i = \frac{1}{2}(X_{i-1} - X_{i+1}), \quad L_i = L(X_i)$$

So we need an algorithm to compute the likelihoods L_i and volumes X_i.

The nested sampling algorithm works like this:

1. Sample n_{live} live points from the prior
2. At each iteration i, find the point with the lowest likelihood. This now becomes a dead point. We record its likelihood L_i^* and remove the dead point from our live points.
3. Sample a new point from the prior, with the constraint that $L(\theta) > L_i^*$
4. Estimate the volume X_i of the likelihood above L_i^*.
5. Estimate $Z = \sum_i L_i^* \Delta X_i$ and iterate from 2. until some convergence criterion on Z is reached.

THE SAME BOXES: 25 LIVE EXPLORERS

Now use a uniform prior on the 100 boxes and let $L(i) = m_i/(1000\,\mathrm{CHF})$.
Each student holds one live point, initially drawn independently from the prior.
Duplicate boxes are allowed.
Remove the point with the lowest L; replace it with a prior draw above that threshold.
Can this keep several peaks in play? What if nobody has found a narrow peak?

The target is still proportional to the money. Now

$$Z = \frac{1}{100}\sum_{i=1}^{100}\frac{m_i}{1000\,\mathrm{CHF}}.$$

Z is average scaled money; $100000\,\mathrm{CHF} \times Z$ is total money.
Live points can represent several peaks. A particular one-box peak is initially missed with probability $(99/100)^{25} \approx 0.778$.
Good constrained exploration is essential. Nested sampling estimates **evidence and weighted posterior samples**; it may use slice sampling internally.

How do we estimate the volumes X_i?
The idea is similar to the Monte Carlo estimation of π at the beginning of the course: we sample an outer volume (the square) and count how many points end up inside the smaller inner volume (the quadrant of the circle).
The ratio of the volumes is then approximated by the ratio of the points inside the inner volume over all the points:

$$\frac{V_{\text{inner}}}{V_{\text{outer}}} \approx \frac{\text{number of samples in } V_{\text{inner}}}{\text{number of samples in } V_{\text{outer}}}$$

How do we estimate the shrinking prior mass?
With $n_{\text{live}} = N$ independent live points uniform in the constrained prior, the shrinkage factor obeys

$$t_i = X_i / X_{i-1} \sim \text{Beta}(N, 1).$$

Hence $\mathbb{E}[t_i] = N/(N+1)$, while $\mathbb{E}[\log t_i] = -1/N$.
A typical log-volume path is $\log X_i \approx -i/N$, starting from $X_0 = 1$. The actual volumes are uncertain; that uncertainty propagates into Z.

The dead points sample the posterior, when weighted appropriately. To see this, consider how an expectation of f with respect to the posterior $p(\theta)$ is defined:

$$\mathbf{E}_p[f] = \int f(\theta) p(\theta) \mathrm{d}\theta = \int f(\theta) \frac{L(\theta)\pi(\theta)}{Z} \mathrm{d}\theta \,.$$

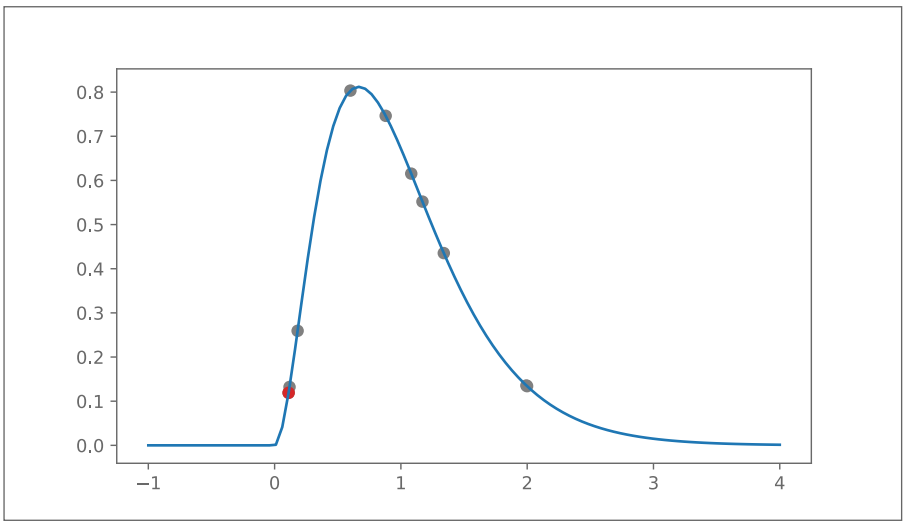
Now write this in terms of $\mathrm{d}X = \pi(\theta)\mathrm{d}\theta$:

$$\mathbf{E}_p[f] = \frac{1}{Z} \int f L \mathrm{d}X \approx \sum_i \frac{w_i L_i}{Z} f_i, \quad f_i = f(\theta_i)$$

So the dead point are posterior samples if they are weighted with weights p_i:

$$p_i = \frac{w_i L_i}{Z} \,, \quad w_i = \frac{1}{2}(X_{i-1} - X_{i+1})$$

Nested sampling is built around estimating the evidence but we get posterior samples for free in the process!



Once Loop Reflect

```
from scipy.special import logsumexp
import tqdm

def sample_nested_sampling(log_likelihood, prior, n_live,
                           tol=0.01, n_max_iter=100000):
    # Sample the initial set of live points from the prior
    live_points = prior.rvs(n_live)
    # Get their log likelihoods
    log_L = log_likelihood(live_points)
    if not np.all(np.isfinite(log_L)):
        raise ValueError("Non-finite log likelihood for some points.")

    # Set up some book-keeping
    log_tol = np.log(tol)

    log_X = [0,]

    dead_points = []
    dead_points_log_L = []

    n_eval = live_points.shape[0]
    drain_live_points = False
    i = 0
    progress = tqdm.tqdm()
    while i < n_max_iter:
        # Find the live point with the lowest likelihood
        idx = np.argmin(log_L)
        # Call the likelihood at this point L^*
        log_L_star = log_L[idx]

        # This lowest likelihood point becomes a dead point
        dead_points.append(live_points[idx])
        dead_points_log_L.append(log_L_star)

        # Estimate the shrinkage of the likelihood volume when removing the
        # lowest-likelihood point
        log_t = -1/n_live
        log_X.append(log_X[-1] + log_t)

        # Check for convergence of the evidence estimate
        if i > 4:
            # Compute the volumes and weights
            X = np.exp(np.array(log_X))
            w = 0.5*(X[:-2]- X[2:])
            # Estimate Z = \sum_i w_i L^*_i
            log_Z = logsumexp(np.array(dead_points_log_L[:-1]), b=w)
            # Estimate the error on Z as the mean of the likelihoods of the
            # live points times the current likelihood volume
            # \Delta Z = X_i \frac{1}{n_{live}}\sum_j L_j
            log_mean_L = logsumexp(log_L, b=1/n_live)
            log_Delta_Z = log_mean_L + log_X[-1]
            # If the estimated error is less than the tolerance, stop sampling
            # new live points for the dead points that get removed
            if log_Delta_Z - log_Z < log_tol:
                drain_live_points = True
                live_points = np.delete(live_points, idx)
                log_L = np.delete(log_L, idx)
                if len(log_L) == 0:
                    break

            progress.set_postfix({"log_Z": log_Z, "n_eval": n_eval, "iter": i})

        # Sample a new live point from the prior with a likelihood higher than
        # L^*
        while not drain_live_points:
```

```
            # Sampling from the whole prior is very inefficient, in practice
            # there are more sophisticed sampling schemes
            new_point = prior.rvs(1).squeeze()
            log_L_new = log_likelihood(new_point)
            n_eval += 1
            if np.isfinite(log_L_new) and log_L_new > log_L_star:
                live_points[idx] = new_point
                log_L[idx] = log_L_new
                break

        i += 1

    # Because the estimate of the volumes is stochastic, we can sample many of
    # them to get the uncertainty on our evidence estimate
    dead_points = np.array(dead_points)
    dead_points_log_L = np.array(dead_points_log_L)
    n_sample = 100
    t_sample = scipy.stats.beta(n_live, 1).rvs((n_sample, len(dead_points_log_L)))
    log_X_sample = np.insert(np.cumsum(np.log(t_sample), axis=1), 0, 0, axis=1)
    X_sample = np.exp(log_X_sample)
    w_sample = 0.5*(X_sample[:, :-2]- X_sample[:, 2:])
    log_Z = scipy.special.logsumexp(dead_points_log_L[:-1], b=w_sample, axis=1)

    return log_Z, dead_points, w*np.exp(dead_points_log_L)[:-1]
```

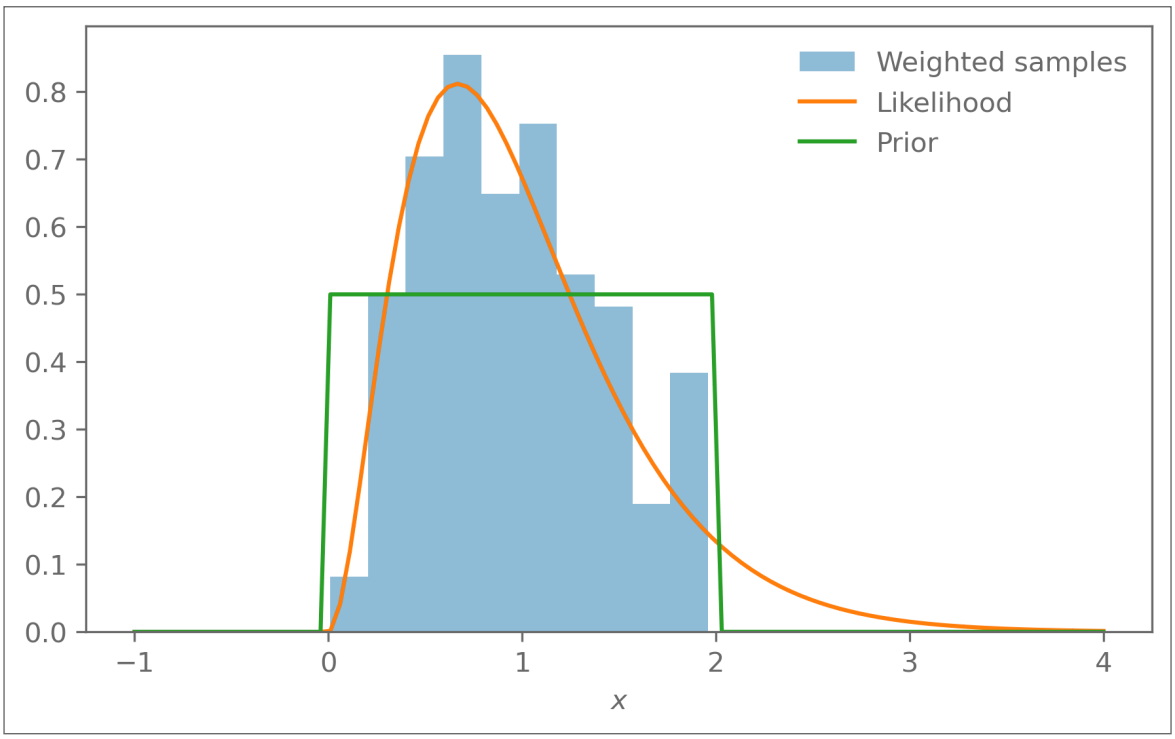
```
log_Z, dead_points, weights = sample_nested_sampling(
    log_likelihood=log_likelihood, prior=prior,
    n_live=100, tol=0.01, n_max_iter=10000)

# In this case we can find the exact evidence by direct integration
import scipy.integrate
log_Z_exact = np.log(
    scipy.integrate.quad(
        lambda x: target_distr.pdf(x)*prior.pdf(x),
        0, 4
    )[0]
)

print(f"Exact log Z: {log_Z_exact:.2f}")
print(f"Nested sampling estimate of log Z: "
      f"{np.mean(log_Z):.2f}±{np.std(log_Z):.2f}")
```

```
0it [00:04, ?it/s, log_Z=-0.813, n_eval=19392, iter=619]

Exact log Z: -0.76
Nested sampling estimate of log Z: -0.81±0.05
```


Weighted samples
Likelihood
Prior
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
−1
0
1
2
3
4
x

CLICKER

What is the primary function of live points in the algorithm?

- They are used to randomly sample new points from the likelihood distribution, which speeds up the convergence.
- They track the likelihood of all previously explored points, ensuring none are revisited during sampling.
- They are used to directly estimate the posterior distribution at each iteration.
- They define the current region of the prior being explored and are iteratively replaced to shrink the likelihood volume.

CHALLENGES WITH NESTED SAMPLING

The big challenge in implementing nested sampling in practice is sampling from the prior with a likelihood constraint. This requires running MCMC or rejection sampling at each step to get a new live point.
If all you care about is posterior samples, then nested sampling can be quite inefficient. Its strength is really the estimation of the evidence, which is important for model comparison.

CLICKER

Recall the Bayes' ratio to compare the probability of two models M_1 and M_2:

$$R = \frac{p(d|M_1)}{p(d|M_2)}$$

With evidences $Z_1 = p(d|M_1)$ and $Z_2 = p(d|M_2)$.
Does the choice of a prior affect the Bayes ratio?

- Yes, because larger prior ranges allow for a broader range of parameter values to contribute to the evidence, leading to a more flexible model comparison.
- No, the prior only affects the normalization of the posterior distribution, not the evidence Z.
- Yes, because the prior volume changes the evidence.
- No, because the priors cancel out in the Bayes ratio.

The evidence is sensitive to the prior volume:

$$Z = \int L(\theta)\pi(\theta)\mathrm{d}\theta$$

Let us assume we have a uniform prior over some volume V: $\pi(\theta) \propto \frac{1}{V}$.
If the likelihood is much more constraining than the prior, the posterior does not change when we change the size of the prior.
But the evidence scales with the prior volume: $Z \propto \frac{1}{V}$. When comparing models, some care must be taken as not to be affected by prior volumes.

DYNESTY

A well-developed package for doing nested sampling is **dynesty**. Other options are **PolyChord** or **ultranest**, or **JAXNS**.
The dynesty and ultranest documentations also have good tutorials and explanations on how nested sampling works.

To allow sampling from the prior, these packages usually require you define a prior transform function that maps samples from the unit (hyper)-cube to samples from your prior using inverse transform sampling.

```python
def prior_transform(u):
    """Transforms samples `u` drawn from the unit cube to samples to those
    from our U(0, 2) prior"""

    return 2*u

def prior_transform_with_scipy(u):
    return scipy.stats.norm(loc=1, scale=2).ppf(u)
```

```python
import dynesty

sampler = dynesty.NestedSampler(
    loglikelihood=lambda x: log_likelihood(x).squeeze(), # Need to add the squeeze so i
    prior_transform=prior_transform,
    ndim=1,
    nlive=100,
)
sampler.run_nested(print_progress=False)

# Show summary of the run
sampler.results.summary()

# We can samples from the distribution
samples = sampler.results.samples_equal()
```

```
Summary
=======
nlive: 100
niter: 291
ncall: 1667
eff(%): 23.455
logz: -0.827 +/-  0.063
```

APPLICATION: IS THE UNIVERSE FLAT?

As the Universe expands, photons lose energy: space expands, which causes the wave length of photons to increase.
Objects further away thus have their spectrum shifted to longer wave lengths: the cosmological redshift.
If we can measure the distance and redshift of an object, we can learn about how the Universe expands, from which we can infer how much dark energy there is, if the Universe is spatially flat, etc.

Here we want to find out if the Universe is better described by a model that assumes that the Universe is spatially flat or one that allows curvature.
Recall that we can use the Bayes' ratio to compare the probability of two models M_1 and M_2:

$$R = \frac{p(d|M_1)}{p(d|M_2)}$$

With nested sampling we can compute the evidences $Z_1 = p(d|M_1)$ and $Z_2 = p(d|M_2)$.

One way to do this to look at Type Ia super novae: due to the astrophysical processes behind them, we know how bright they are intrinsically. This allows us to estimate the distance to a Type Ia super nova. This is a special property, usually we do not know how far objects are away!

Given a redshift z and a cosmological model, we can compute the luminosity distance $d_L(z)$:

$$d_L(z) = (1+z) D_M(z)\,,$$

where D_M is the transverse comoving distance. For a flat geometry, $D_M(z) = c\int_0^z \mathrm{d}z'/H(z')$. Neglecting radiation,

$$H(z)^2 = H_0^2\left(\Omega_m(1+z)^3 + \Omega_\Lambda + (1-\Omega_m-\Omega_\Lambda)(1+z)^2\right).$$

Here H_0 is the Hubble constant (the expansion rate of the Universe today), Ω_m the amount of matter in the Universe, and Ω_Λ the amount of dark energy.

For historical reasons, units in astronomy are weird. Instead of using the luminosity distance as the quantity to compare with data, we predict the apparent magnitude $m(z) = \mu(z) + M$:

$$m(z) = 5\log_{10}\left(\frac{d_L(z)}{10\,\mathrm{pc}}\right) + M\,,$$

where μ is the distance modulus and M is an absolute-magnitude calibration parameter that we will marginalise over.

For the data we use the Pantheon+ sample (Brout+2022, **https://arxiv.org/abs/2202.04077**). The release contains 1701 light curves of 1550 distinct Type Ia supernovae; this classroom example uses a selected subset. We simplify our analysis a little bit here but the full setup is not that much more complicated.
The data are observations of the corrected apparent magnitudes $\hat{m}_i$ at redshifts z_i. The data are correlated, so we need a multivariate Gaussian likelihood:

$$\hat{\boldsymbol{m}} \sim \mathcal{N}(\boldsymbol{m}(\vec{z}), \Sigma)\,,$$

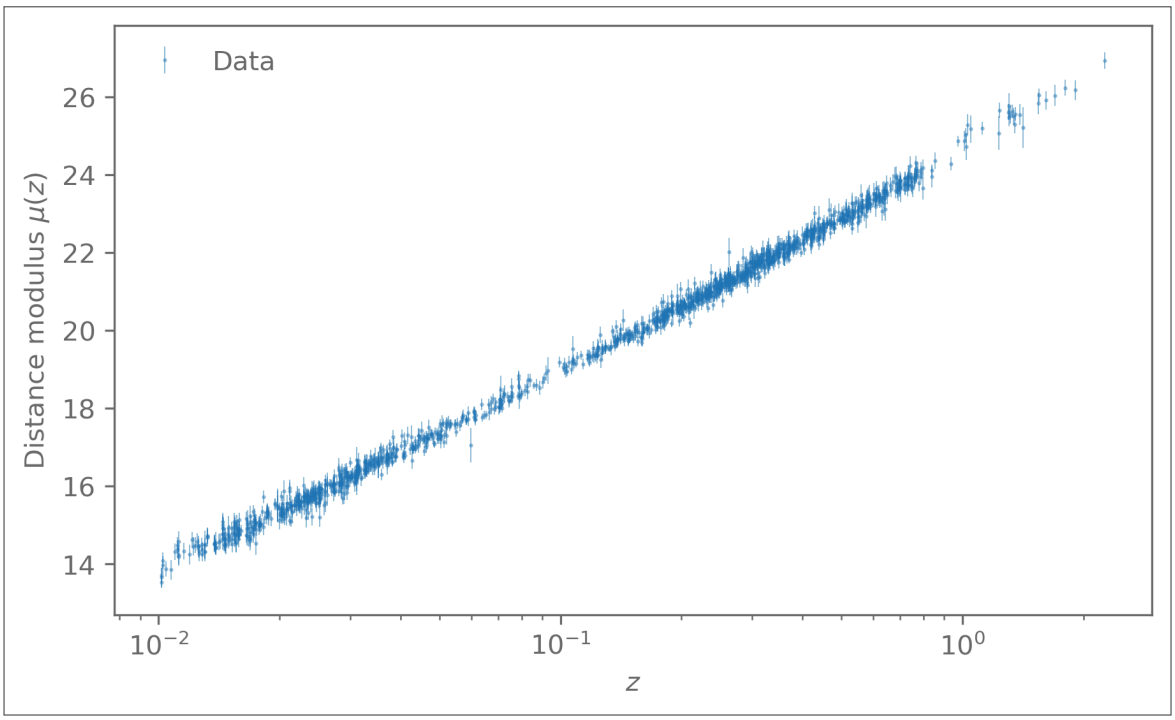
where $\boldsymbol{m}(\vec{z})$ is the predicted apparent magnitude from the last slide.

The model helper is in `course_tools/src/bayesian_stats_course_tools/pantheon.py`; the data are in `assets/dark_energy_SN/`:

```
from bayesian_stats_course_tools.pantheon import PantheonSH0ESLikelihood

# This holds the data and takes care of the modelling
pantheon_sh0es = PantheonSH0ESLikelihood(
    data_file_name=repo_root / "assets" / "dark_energy_SN" / "pantheon_sh0es.npz",
)

# Pull out data for our simplified analysis
z_data = pantheon_sh0es.z_CMB
data = pantheon_sh0es.magnitude_data
data_error_bar = pantheon_sh0es.magnitude_data_error
inv_covariance = np.linalg.inv(pantheon_sh0es.covariance_no_calibrator)
```


Data
Distance modulus $\mu(z)$
26
24
22
20
18
16
14
10^{-2}
10^{-1}
10^{0}
z

Implement the physics model. We use the `astropy` library to compute the luminosity distance.

```python
def compute_distance_modulus(cosmology, z):
    luminosity_distance = cosmology.luminosity_distance(z)
    luminosity_distance = (luminosity_distance/astropy.units.Mpc).value

    mu = 5.0*np.log10(luminosity_distance*1e6/10)
    return mu

def LCDM_distance_modulus_model(theta, z):
    H_0, Omega_m, Omega_de, M = theta
    cosmology = astropy.cosmology.LambdaCDM(H0=H_0, Om0=Omega_m, Ode0=Omega_de)

    prediction = compute_distance_modulus(cosmology, z)
    prediction += M

    return prediction
```

We also look at another model where we assume the Universe is spatially flat. This is the case when $\Omega_m + \Omega_\Lambda = 1$. We then want to find out if the data prefer the general model or the flat model.

```
def flat_LCDM_distance_modulus_model(theta, z):
    H_0, Omega_m, M = theta
    cosmology = astropy.cosmology.FlatLambdaCDM(H0=H_0, Om0=Omega_m)

    prediction = compute_distance_modulus(cosmology, z)
    prediction += M

    return prediction
```

Define our priors. In our simplified analysis we cannot constrain H_0, we use the results from another experiment as our prior.

```
# From the SH0ES analysis Riess+2022 (https://arxiv.org/abs/2112.04510)
H_0_prior = scipy.stats.norm(loc=73.04, scale=1.04)

# scipy.stats.uniform defines the interval as [loc, loc+scale]
Omega_m_prior = scipy.stats.uniform(loc=0.1, scale=0.8)
Omega_de_prior = scipy.stats.uniform(loc=0.1, scale=0.8)
M_prior = scipy.stats.uniform(loc=-20, scale=2)
```

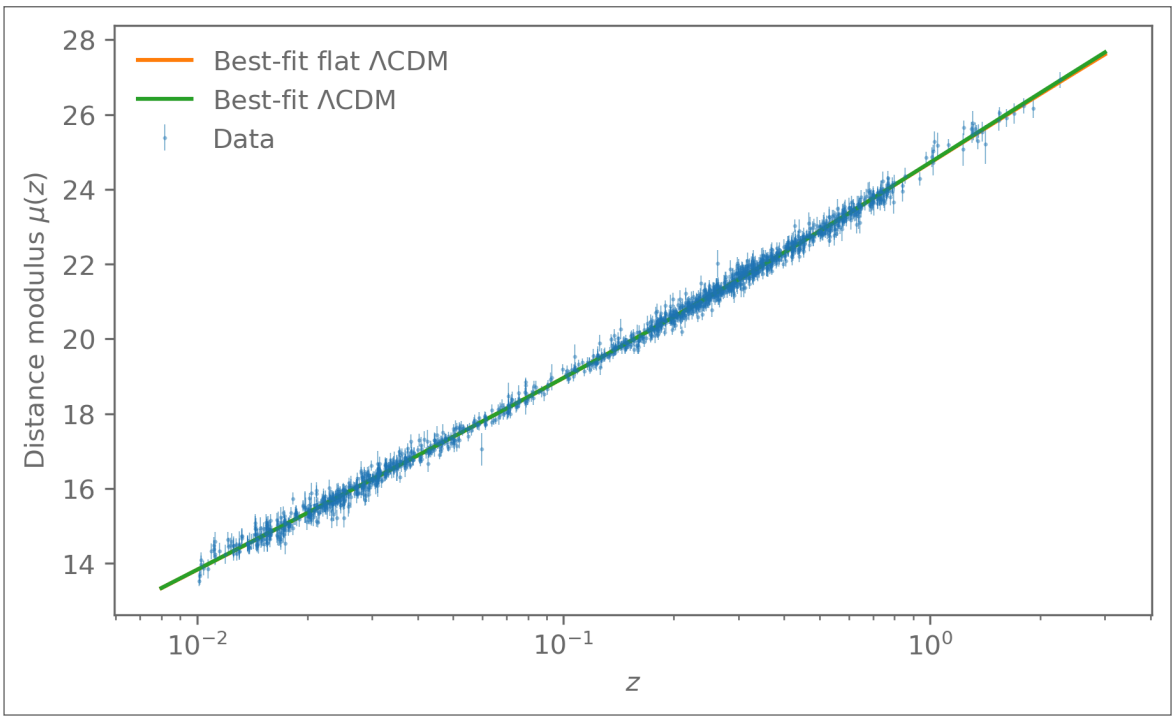
Finally the multivariate Gaussian likelihood.

```
def make_likelihood(model):
    def log_likelihood(theta):
        r = data - model(theta, z=z_data)
        return -0.5 * r @ inv_covariance @ r

    L = np.linalg.cholesky(pantheon_sh0es.covariance_no_calibrator)

    def sample_from_likelihood(theta):
        # scipy.stats.multivariate_normal is VERY slow for the
        # large covariance we have, so we sample it ourselves
        mu = model(theta, z=z_data)
        sample = L @ np.random.normal(size=mu.size) + mu
        return sample

    return log_likelihood, sample_from_likelihood
```


Best-fit flat ΛCDM
Best-fit ΛCDM
Data
Distance modulus $\mu(z)$
z
28
26
24
22
20
18
16
14
10^{-2}
10^{-1}
10^{0}

Both models fit the data very well. Can we still say which one we should prefer?
Recall that we can use the Bayes' ratio to compare the probability of two models M_1 and M_2:

$$R = \frac{p(d|M_1)}{p(d|M_2)}$$

With nested sampling we can compute the evidences $Z_1 = p(d|M_1)$ and $Z_2 = p(d|M_2)$.

Set up the priors for `dynesty`:

```
def prior_transform_flat_LCDM(u):
    x = np.array([
        H_0_prior.ppf(u[0]),
        Omega_m_prior.ppf(u[1]),
        M_prior.ppf(u[2])
    ])

    return x

def prior_transform_LCDM(u):
    x = np.array([
        H_0_prior.ppf(u[0]),
        Omega_m_prior.ppf(u[1]),
        Omega_de_prior.ppf(u[2]),
        M_prior.ppf(u[3])
    ])

    return x
```

And run `dynesty` to get estimates of the evidence. We get samples from the posterior as well during this.

```
sampler_flat_lcdm = dynesty.NestedSampler(
    loglikelihood=log_likelihood_flat_LCDM,
    prior_transform=prior_transform_flat_LCDM,
    ndim=3,
    nlive=100,
)
sampler_flat_lcdm.run_nested(print_progress=True)

sampler_flat_lcdm.results.summary()
```

```
1070it [01:23, 12.78it/s, +100 | bound: 71 | nc: 1 | ncall: 14410 | ef
f(%):  8.176 | loglstar:   -inf < -720.845 <    inf | logz: -729.220
+/-  0.270 | dlogz:  0.001 >  0.109]
Summary
=======
nlive: 100
niter: 1070
ncall: 14310
eff(%):  8.176
logz: -729.220 +/-  0.321
```

To get an estimate of the uncertainty of the Bayes' ratio, we use a tool from `dynesty` to resample our runs to get a set of evidence estimates.

```
log_R = []
for i in range(100):
    log_z_flat_lcdm = dynesty.utils.jitter_run(sampler_flat_lcdm.results).logz[-1]
    log_z_lcdm = dynesty.utils.jitter_run(sampler_lcdm.results).logz[-1]
    log_R.append(log_z_flat_lcdm - log_z_lcdm)

log_R = np.array(log_R)

R_mean = np.mean(np.exp(log_R))
R_std = np.std(np.exp(log_R))
print(f"Bayes' ratio: {R_mean:.2f} ± {R_std:.2f}")
```

```
Bayes' ratio: 0.97 ± 0.36
```

The stored runs give no clear evidence preference between the flat model and the curved model under these priors. Posterior model odds also depend on the prior model odds.

PROJECT IDEAS

- Look deeper into one of the sampling methods. For example, how do the more sophisticated algorithms in `emcee`, `zeus`, or `dynesty` work?
- Do a full analysis of the super nova data set, following the Bayesian data analysis workflow:
    1. Build the model (that's already done for you), check that the model is reasonable with prior predictive checks
    2. Fit the model (MAP, posterior). Analyse the posterior distribution.
    3. Check the model, compare between models

General data analysis projects:
No original research is expected. If you use real data, it is probably better to follow and reproduce an existing analysis of these data.
Even if it is a published paper, there is a very good chance that you can improve the statistical analysis by using the methods from this course!
Focus on the statistics, not the messiness of real data!

EXERCISE

Implement your own nested sampling or slice sampling routine. Or both if you are ambitious.
Use both emcee and dynesty on the 2D Gaussian from the Metropolis-Hastings exercise.
Go back to the line-fitting exercise. Use nested sampling to find the evidences for a linear and a quadratic model. Which model is preferred by the data?